

КОД БЕЗ КОММЕНТАРИЕВ:

```
from itertools import *

def fun(x, y, z, w):
    return not((x <= (not w)) and z) and not(w <= z) and (x <= (not z))

for per in permutations('xyzw'):
    for a, b, c, d, e in product([0, 1], repeat=5):
        tab = [(1, 0, a, 0, 1), (1, 0, b, c, 0), (d, 1, e, 1, 0)]
        rez = []
        for t in tab:
            rez.append(fun(**dict(zip(per, t))))
        if len(set(tab)) == 3 and rez == [1, 0, 0]:
            print(*per, tab)
            break
```

=====

КОД С КОММЕНТАРИЯМИ:

```
from itertools import *

# Импортируем функции для работы с функциями:
# permutations - для перебора всех перестановок букв хуэв
# product - для перебора всех возможных комбинаций 0 и 1

def fun(x, y, z, w):
    # Определяем логическую функцию fun от 4 переменных
    return not (not (x <= (not w)) and z) and not (w <= z) and (x <= (not z))
    # Вычисляем и возвращаем значение сложного логического выражения:
    #  $\neg (\neg (x \rightarrow \neg w) \wedge z) \wedge \neg (w \rightarrow z) \wedge (x \rightarrow \neg z)$ 

for per in permutations('xyzw'):
    # per - очередной вариант переставленных букв, например ('x','y','z','w') или ('x','w','z','y')
    # Всего 4! = 24 возможных перестановки
    for a, b, c, d, e in product([0, 1], repeat=5):
        # a,b,c,d,e - это неизвестные значения в таблице истинности (обозначены как ?)
        # product([0,1], repeat=5) генерирует все возможные комбинации из 5 цифр 0 или 1
        # Всего 2^5 = 32 комбинации
        tab = [(1, 0, a, 0, 1), (1, 0, b, c, 0), (d, 1, e, 1, 0)]
        # Создаем таблицу из 3 строк (каждая строка - кортеж из 5 значений)
        # В каждой строке:
        # - цифры 0 и 1 - это известные значения из условия
        # - буквы a,b,c,d,e - это неизвестные значения, которые мы перебираем
        rez = []
        # Создаем пустой список для хранения результатов вычисления функции
        for t in tab:
            # t - очередная строка таблицы, например (1, 0, a, 0, 1)
            rez.append(fun(**dict(zip(per, t))))
            # Разбираем по шагам:
            # 1. zip(per, t) - соединяет буквы (хуэв) и их значения (0/1) в пары-кортежи:
            #    например, при per=('x','y','z','w') и t=(1,0,a,0) получим:
            #    [('x',1), ('y',0), ('z',a), ('w',0)]
            # 2. dict(...) - превращает список пар в словарь: {'x':1, 'y':0, 'z':a, 'w':0}
            # 3. **dict - распаковывает словарь в аргументы функции: fun(x=1, y=0, z=a, w=0)
            # 4. Результат вычисления функции добавляем в список rez
        if len(set(tab)) == 3 and rez == [1, 0, 0]:
            # Проверяем два условия:
            # 1) len(set(tab)) == 3 - все строки таблицы разные (нет повторяющихся)
            #    set(tab) превращает список строк в множество, убирая дубликаты
            # 2) rez == [1,0,0] - значения функции совпадают с последним столбцом таблицы
            #    (по условию задачи в последнем столбце должны быть значения 1,0,0)
            print(*per, tab)
            # Если оба условия выполнены - нашли решение
            # Выводим: перестановку букв и соответствующую таблицу истинности для проверки
            # *per распаковывает перестановку, убирая скобки, кавычки и запятые: x y z w
            break
    # Так как решение для текущей перестановки уже найдено,
    # дальше с ней работать не нужно - выходим из внутреннего цикла
```